ENGINEERING PROJECT / 2026

StegoLens

Steganographic Malware Detection in AI Model Files

Research tool for identifying hidden-data indicators and comparing model files

Project team

Pranav Khandelwal – A027

Jaimil Maniar – A029

Nakul Maurya – A030

Rishit Nanavati – A034

Purpose and problem

An AI model file contains many learned numbers. Data can be hidden inside those numbers—a technique called steganography. A reviewer needs a way to look for unusual signs before deciding how to handle an unfamiliar model file.

StegoLens inspects supported files locally, compares available evidence and explains what was actually examined. It does not run an uploaded model during an ordinary scan or certify that a file is safe. [1, 2]

Important boundary The title names the research problem. The tool identifies indicators for review; it does not prove that hidden data are malware or detect every backdoor.

Established software baseline: v0.5.0, evidence schema 1.1, repository snapshot 8ddce8e. Prepared 27 October 2026. The existing code and repository retain the legacy T04-Scnr name for compatibility. Measurements below retain their recorded source provenance; the project display name is StegoLens.

1. Problem definition and our approach

Three different concerns—not one verdict

A suspicious model file can raise several questions. StegoLens keeps them separate so that one result is not mistaken for another. [1–3]

Concern	What it means in plain language
Hidden data in weights	Steganography stores extra information in a model's learned numbers. A hidden message may be harmless; detecting a pattern does not prove malware.
Malware or unsafe loading	Malware is harmful software. Some model-loading formats or paths can execute code. A static warning and proof of a working malicious program are different claims.
Behavioral backdoor	A model can behave normally on ordinary inputs but make an attacker-chosen mistake when a special trigger appears. Looking at stored numbers alone does not prove or exclude this behavior.

Why a simple "safe / infected" score is not enough

Unusual weights can result from normal training. A difference from another model may be expected. A scan may inspect only part of a file, and some optional checks may be unavailable. Combining these into a reassuring safety score would hide uncertainty.

Our approach: inspect, separate, explain

- Inspect locally. Identify the file and supported structure without executing arbitrary uploaded model code.
- Measure bounded samples. Examine selected regions of stored numbers rather than promise complete inspection.
- Compare independent evidence. Keep static warnings, numerical indicators, detector votes and reference differences separate.
- Save what happened. Keep results, sample coverage and missing checks in an unchanging evidence record.
- Explain in readable language. Show the meaning and denominator of each number in a dashboard or offline report.

A controlled test is not a malware test

The exploratory detector benchmark adds harmless text or seeded random bytes to selected weight bits. It tests whether the numerical features recognize those modifications. "No added benchmark data" does not mean "safe model," and a detector vote is not the probability of malware. [3]

2. Architecture: what the parts do

The diagram shows how information moves through the established application. Optional checks can be unavailable; that absence stays in the result. [1, 2, 4]

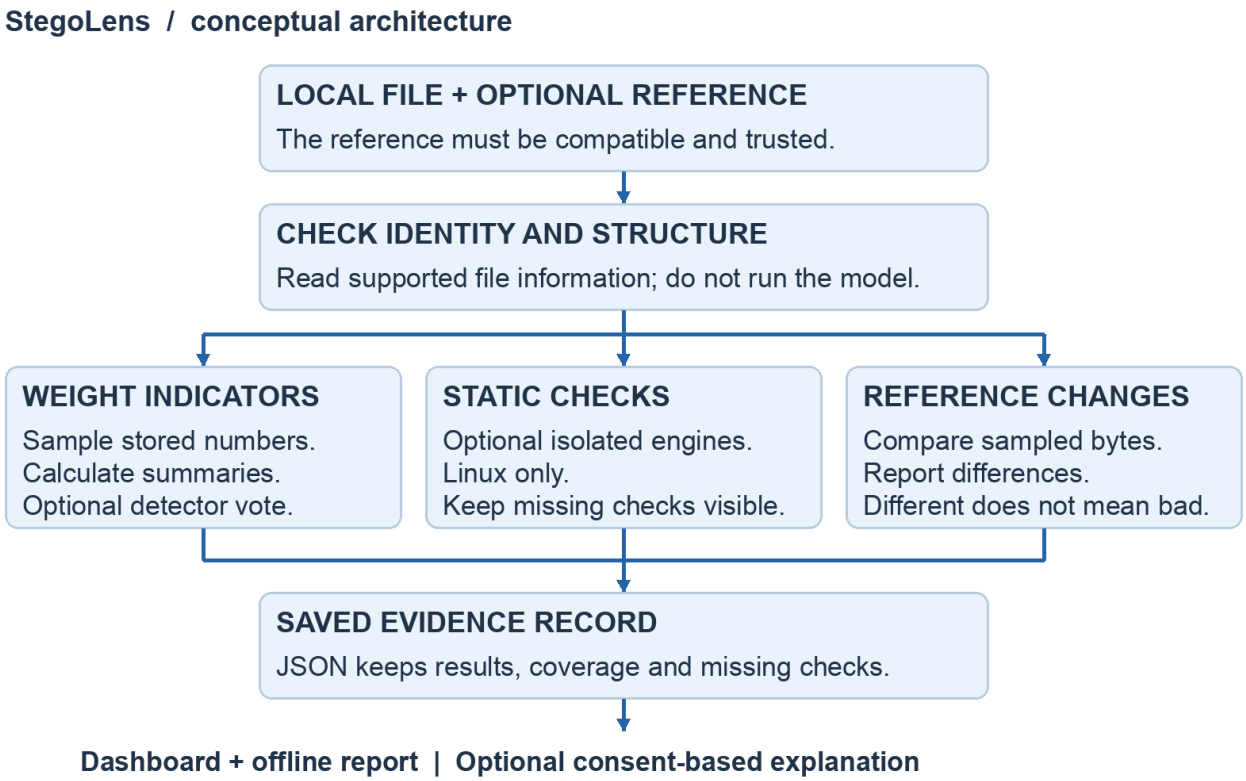


Figure 1. Conceptual StegoLens architecture, newly drawn from the project's documented data flow [2]. It is not a screenshot. Windows deliberately leaves Linux-only workers unavailable rather than using an unsafe fallback.

Reading the diagram

Input checking records the file's identity and inventory. The three middle paths answer different questions: are sampled numbers unusual, are static warnings present, and how does the file differ from a reference?

The saved record is the source of truth. The dashboard and offline HTML/PDF reports present that record; exporting a report does not scan the file again or run the model.

An explanation can be generated offline. Optional Groq use requires a key and explicit consent for a sanitized, limited evidence bundle. It selects validated application-written explanation cards—not unrestricted diagnostic text—and cannot change the scan evidence. [4]

3. Working: from upload to result

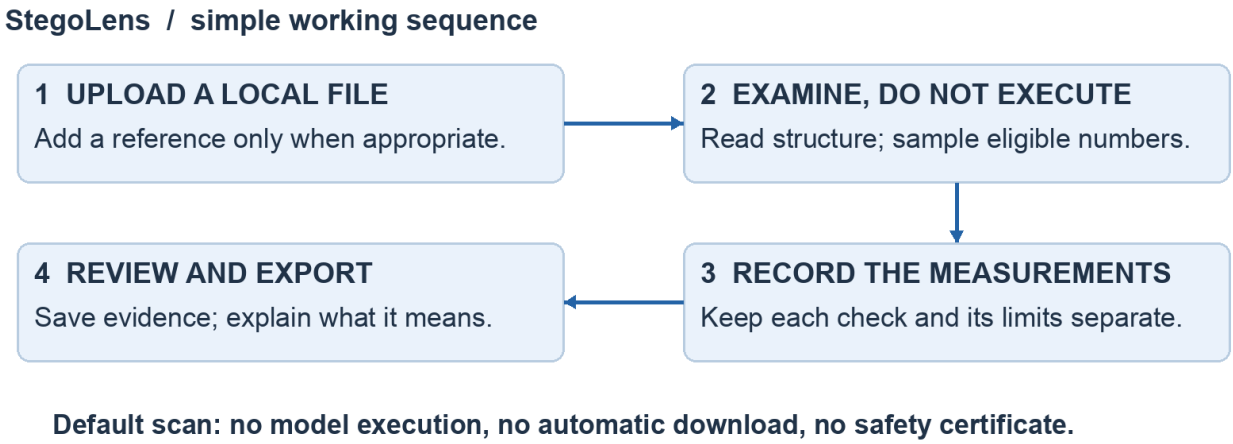


Figure 2. Simple conceptual workflow. The normal scan examines stored data; the separate prepared demonstration on page 5 explicitly runs known models.

The full process, in plain language

1. Choose the file. The user supplies a local model file and, optionally, a trusted compatible reference. The default scan does not download a model. A file hash acts as an exact-byte fingerprint—not a certificate of trust. [1, 2]

2. Read supported structure. For safetensors files, the tool checks the header and lists the arrays of learned numbers, called tensors. It records their sizes and number formats. It does not train, repair or execute the uploaded model.

3. Examine selected regions. The scanner samples bounded windows of eligible 32-bit weights (F32). It summarizes their average, spread, magnitude and byte patterns. Separate 16-bit formats (F16/BF16) receive descriptive analysis; they do not silently reuse the 32-bit detector rules. [1, 3]

4. Add optional evidence. A small Random Forest detector—many decision trees—scores the numerical summaries using data-only JSON trees. It is the detector that runs, not the uploaded model. Optional Linux-isolated static engines add their own status. A reference comparison counts changed sampled bytes; change alone does not mean maliciousness.

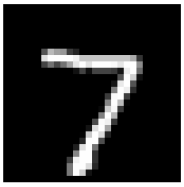
5. Save and explain. An unchanging JSON record stores measurements, thresholds, coverage and skipped checks. The user reads or exports that evidence. For example, “92.09% of recorded total F32 elements sampled” means that population was examined—not that the model is 92.09% safe. [1, 2, 5]

What happens to the model? Ordinary scanning inspects its bytes and numbers. It does not modify the uploaded weights, remove a backdoor or produce a certified clean replacement.

4. Results: what the evidence shows

StegoLens / recorded known-trigger example

Normal image: true digit 7



Earlier checkpoint (epoch 1): 7

Published poisoned checkpoint (epoch 2):

Prediction: 7

Same image + four white trigger pixels



Earlier checkpoint (epoch 1): 7

Published poisoned checkpoint (epoch 2):

Prediction: 8

Figure 3. Actual recorded CPU predictions for the first MNIST test image, drawn from the supplied samples.json. Four known white trigger pixels change epoch 2's prediction from 7 to 8. Epoch 1 is an earlier reference, NOT an independently clean-trained control. This example illustrates behavior, not the scanner's detection accuracy. [6, 7]

Separate known-trigger experiment: all 10,000 test images

Published checkpoint	Normal accuracy	Known-trigger target hits
Epoch 1: earlier reference	94.31% (9,431 / 10,000)	0.41% (41 / 10,000)
Epoch 2: poisoned checkpoint	96.64% (9,664 / 10,000)	94.89% (9,489 / 10,000)

These prior Linux checks use the exact published checkpoints and documented four-pixel trigger. The target is the next digit, with 9 wrapping to 0. Each percentage uses all 10,000 test images. The result shows that high ordinary accuracy can coexist with a known backdoor; it is not app detection accuracy or a test of unknown triggers. [6, 7]

What the static investigation recorded

A historical scan sampled 56,692 of 61,562 total recorded F32 values (92.09%); 4 of 17 windows met the recorded detector threshold. Reference comparison found 143,865 changed bytes out of 185,320 compared (77.63%). These describe sampled data, not a percentage of malware. [5]

In the controlled harmless-byte test, 76.21% of flagged windows matched the added-data label, and 73.83% of labeled windows were found. Its 384 test windows came from just one held-out source family—not 384 independent models. This is not broad malware-detection accuracy. [3, 5]

5. Conclusion, limits and references

StegoLens makes model-file investigation more understandable: identify a file, collect separate indicators, compare evidence and explain the result. Its strongest contribution is traceability—showing what was examined and what remains unknown—not a universal security verdict.

Limits and next steps

- Sampling is partial. Small arrays, unsupported formats, different number types and missing engines can leave gaps. No finding must never be rewritten as “clean” or “safe.”
- Hidden-data indicators do not prove malware. Reference differences do not prove intent. A known-trigger experiment does not discover every possible backdoor.
- The benchmark is small. Many windows from one source are not many independent models. Future work needs broader independent model pairs and retained false positives and misses.

Regression tests cover scan safety, reporting and dashboard behavior; platform-specific limitations remain explicit. Software tests check the implementation, not the accuracy of malware detection. [8]

Sources and provenance

Repository documents use snapshot 8ddce8e. Existing T04-Scnr URLs remain compatibility/source references. This report does not rerun the full experiments or make a new software-release claim.

[1] Overview and supported features
https://github.com/nakulmaurya/T04-Scnr/blob/8ddce8e/README.md

[2] Architecture and trust boundaries
https://github.com/nakulmaurya/T04-Scnr/blob/8ddce8e/docs/ARCHITECTURE.md

[3] Controlled harmless-byte benchmark and its limits
https://github.com/nakulmaurya/T04-Scnr/blob/8ddce8e/docs/BENCHMARK.md

[4] Optional evidence explanation and consent
https://github.com/nakulmaurya/T04-Scnr/blob/8ddce8e/docs/EVIDENCE_ASSISTANT.md

[5] Historical scan/benchmark example; supplied original report
https://github.com/nakulmaurya/T04-Scnr/blob/8ddce8e/docs/PROJECT_REPORT.md

[6] Exact demo method, trigger and full-test-set results
https://github.com/nakulmaurya/T04-Scnr/blob/8ddce8e/docs/DEMOS.md

[7] Publisher’s MNIST checkpoints
https://huggingface.co/authentrics/mnist-badnets-poisoned

[8] v0.5.0 software/platform validation and skip scope
https://github.com/nakulmaurya/T04-Scnr/blob/8ddce8e/docs/VALIDATION_V050.md

Figure 3 uses recorded sample 0 of the first 12 canonical MNIST test inputs, selected in order without filtering (samples.json, source commit 8ddce8e; 27 October 2026). Exact model/data hashes accompany that record. The illustration is newly laid out; it is not a new accuracy estimate. Figures 1–2 are conceptual diagrams, not UI screenshots.